RADAR

Plattform für Forderungsrisiko-Scoring & Kreditentscheidungsunterstützung

Technische Dokumentation · v1.0

DynamicAI · 2026

Radar — Technische Dokumentation von A bis Z

🎯 Individuell anpassbar für Ihr Unternehmen

Radar ist eine Open-Source-Plattform (AGPL-3.0) und ist **an die Geschäftsregeln, das Branchenprofil und die Risikobereitschaft jedes Unternehmens anpassbar**. Scoring-Formeln, Gewichtungen, Branchenrisikofaktoren, Makro-Annahmen (Zins, Inflation), Zahlungsziel-Stufen und Entscheidungslogik sind vollständig konfigurierbar.

Kontaktieren Sie DynamicAI für **Unternehmensintegration, kundenspezifische Modulentwicklung, On-Premise-Bereitstellung und Schulungen**: *osmanedembele6361@gmail.com*

> Plattform für Forderungsrisiko-Scoring und Kreditentscheidungsunterstützung. Letzte Aktualisierung: 2026-05-03

---

1. Projektübersicht

Radar ist eine Anwendung für **Kredit-Scoring / Forderungsrisikobewertung**, die sich an KMU und Konzernkunden richtet. Sie positioniert sich als das interne (private) Gegenstück zu einem Findeks-ähnlichen Rating-Service.

Kern-Workflow: 1. Der Nutzer legt einen Kunden (Konto) an 2. Erfasst Aging- und Kreditantragsdaten oder importiert sie per Excel 3. Das System erzeugt eine **Bonitätsnote** auf Basis von Zahlungsdisziplin, Finanzkennzahlen (Z-Score, F-Score, DSCR, ICR), Branchenrisikofaktoren und Makrobedingungen 4. Monte-Carlo-Simulation + benannte Stresstests berechnen die Szenarienverteilung 5. Es wird ein mehrsprachiger (TR/EN/ES/DE) detaillierter PDF/HTML-Bericht erstellt

---

2. Technologie-Stack

Schicht	Technologie
Backend	Python 3.11, **Flask 3.0** (Blueprints)
ORM	**SQLAlchemy 2.0** (scoped_session)
DB	PostgreSQL (prod) / SQLite (dev) — Dual-Target
Auth	flask-login + **Authlib** (Google OAuth) + itsdangerous (Mail-Verify)
Mail	flask-mail (SMTP)
i18n	**flask-babel 4.x** (locale_selector-Pattern)
Templating	Jinja2 + Bootstrap 5.3 + Bootstrap Icons
API Docs	**Flasgger** (OpenAPI/Swagger)
Async	Celery + Redis (geplant, noch nicht aktiv)
Container	Docker + docker-compose, `network_mode: host`
WSGI	gunicorn 4 Worker
Reverse Proxy	nginx — `technodai.com/radar/` → `127.0.0.1:5001`
Excel	pandas + openpyxl

3. Architektur (Übersicht)

```
┌──────────────────────────────────────────────────────┐
│  Browser (TR/EN/ES/DE)                               │
└──────────────────────────────────────────────────────┘
                        │
                        │ https://technodai.com/radar
                        ▼
                  ┌──────────┐
                  │  nginx   │  TLS, prefix routing
                  └──────────┘
                       │
                       │ proxy_pass :5001
                       ▼
              ┌──────────────────┐
              │  gunicorn × 4    │  Docker (host net)
              │   ↓              │
              │  Flask app       │
              │   ├ blueprints/  │  auth, main, customer,
              │   │              │  scoring, admin, api
              │   ├ enterprise/  │  i18n, RBAC, audit, webhooks
              │   └ analytics/   │
              └──────────────────┘
                        │
                        │
            ┌───────────┼──────────┐
            ▼           ▼          ▼
       PostgreSQL   SMTP      Google OAuth
       (advisory   (verify     (login)
        locks)     mails)
```

---

4. Modulkarte

4.1 Kern

Datei	Verantwortung
`app.py`	Flask-Factory (`create_app`), Blueprint-Registrierung, `init_db()` , Init Enterprise-Features
`database.py`	SQLAlchemy-Modelle, Dual-DB-Engine, leichte ALTER-Migration, **Postgres advisory lock** (Worker-Sicherheit)
`extensions.py`	`login_manager` , `mail` , `ts` (URLSafeTimedSerializer) — modulweite Singletons
`credit_scoring.py`	Klasse `CreditScorer` — Score-Berechnung, Experten-Bewertungsnotizen, Monte Carlo + benannte Stresstests
`aging_analyzer.py`	Erzeugt Verzugskennzahlen aus Aging-Datensätzen
`excel_import.py`	Massenimport Kunden/Datensätze (pandas)
`currency.py`	TRY-Formatierung, FX-Konvertierungs-Helfer
`logger.py`	Strukturiertes Logging
`translations.py`	4-Sprachen-Wörterbuch (TR/EN/ES/DE) — flat dict, ~70 KB

4.2 Routes (Blueprints)

Blueprint	Pfad	Verantwortung
`auth`	`/login` , `/register` , `/logout` , `/google-login` , `/verify/<token>`	Benutzer-Authentifizierung
`main`	`/` , `/nedir` , `/guvenlik` , `/set_language/<lang>`	Startseite, statischer Inhalt, Sprachwechsel
`customer`	`/musteri/...`	CRUD + Detail
`scoring`	`/skor/...` , `/rapor/<id>`	Score-Berechnung, Bericht-Render
`admin`	`/admin/...`	Einstellungen, Benutzerverwaltung, Audit-Log
`api`	`/api/v1/...`	REST-Endpunkte (per Flasgger dokumentiert)

4.3 Enterprise-Schicht (Phase 8-11)

Datei	Verantwortung
`enterprise.py`	`init_enterprise_features(app)` — RBAC, Audit, Multi-Tenant-Init
`i18n_utils.py`	flask-babel 4.x Locale-Selector
`analytics.py`	Tenant-bezogene Metrik-Aggregation
`api_docs.py`	Flasgger-Swagger-UI-Konfiguration
`webhooks.py`	Outbound-Webhook-Versand (Slack, Discord, custom)

---

5. Datenmodell

```
Tenant 1──* User 1──* Customer 1──* AgingRecord
                                  │
                                  ├──* CreditRequest
                                  └──* CreditScore (history)

User *──* Role  (UserRole pivot)
Tenant 1──* AuditLog
```

Wichtig: - `tenant_id` ist in jeder Tabelle `nullable=True default=1` — Altdaten fallen auf den Default-Tenant - Kunden mit `is_sample=True` sind für alle Nutzer sichtbar (Demo-Daten) - `CreditScore` ist versioniert — jede Berechnung erzeugt eine neue Zeile, Historie wird nicht gelöscht

---

6. Deployment-Architektur

Prod (technodai.com)

```
~/apps/radar/                  # git clone
├── docker-compose.yml         # nur lokales Port-Binding (5001)
├── .env                       # SECRET_KEY, DB_URL, OAuth, SMTP
└── data/                      # SQLite-Fallback + Uploads (Volume)
```

- Container `radar_v1_app`, `restart: always`, Host-Network
- Postgres-DB `radar`, `pg_advisory_lock(7382001)` verhindert die Init-Race der 4 Worker
- nginx-vhost: Prefix `/radar/` mit `proxy_pass http://127.0.0.1:5001/`

Lokal (dev)

- Selbe Compose-Datei (Desktop-Symlink), Port `8005`
- Fällt auf SQLite zurück, läuft ohne separate DB

Deployment-Ablauf

```
lokale Änderung
   ↓ git commit + push
   ↓ ssh prod
cd ~/apps/radar && git pull
sudo docker compose up -d --build
   ↓ Smoke-Test
curl -I https://technodai.com/radar/
```

---

7. Entwicklungs-Workflow

1. **Branch**: kleine Commits auf `main`. Topic-Branches für größere Features.
2. **Migration**: `database.py:_run_lightweight_migrations()` nutzt das Pattern `ALTER TABLE IF NOT EXISTS` — Alembic für das kleine Schema bewusst weggelassen.
3. **Seed**: `seed_sample_data.py` — Demo-Kundensatz für Neuinstallationen.
4. **Tests**: `tests/security_smoke.py` läuft in CI; `integrity_check.py` ist manuell.
5. **i18n**: Neue Strings müssen in allen 4 Blöcken von `translations.py` ergänzt werden. CI erzwingt Schlüssel-Parität.
6. **Commit-Message**: `<type>(<scope>): <Zusammenfassung>` — `feat`, `fix`, `chore`, `refactor`.

---

8. Endbenutzer-Ablauf

Schritt	Seite	Aktion
1	`/register`	Anmeldung per E-Mail/Passwort oder Google → Mail-Verifikation
2	`/`	Kundenliste — neuen Kunden hinzufügen
3	`/musteri/yeni`	Konto-Infos (Branche, Eigenkapital, Umsatz, etc.)
4	`/musteri/<id>`	Aging-Zeile + Kreditantrag erfassen
5	`/musteri/<id>/import`	Massenimport per Excel
6	`/skor/<id>`	Score berechnen — Admin-Einstellungen (Zins, Inflation) werden angewendet
7	`/rapor/<id>`	Detailbericht — Führungsbewertung, Szenarien, benannte Stresstests
8	`/admin`	Einstellungen, Benutzerverwaltung, Audit-Log

---

9. Roadmap — Was folgt

9.1 Kurzfristig (1-2 Wochen)

- [ ] **Erweiterung des Zahlungsziel-Moduls**: heute statisch gestaffelt (`vade_15..90`); dynamische Berechnung nach Zahlungsdisziplin-Band (DSO + Branchenmedian)
- [ ] **PDF-Export**: WeasyPrint-Integration für `?format=pdf` im Bericht
- [ ] **Unit-Test-Skelett**: `pytest` + `pytest-flask` , ≥70 % Coverage in `credit_scoring`
- [ ] **Audit-Log-UI**: filterbare Tabelle im Admin-Panel

9.2 Mittelfristig (1-2 Monate)

- [ ] **Celery-Aktivierung**: Excel-Import + Score-Batch raus aus dem Request-Thread (große Dateien laufen heute in den Timeout)
- [ ] **Webhook-Event-Erweiterung**: `score.calculated` , `customer.high_risk` , `request.spike`
- [ ] **Rate-Limiting (per-Tenant API)**: Redis-Backend-Quote für Login/Register aktiv; per-Tenant `/api/v1/*` -Quote noch offen
- [ ] **Branchen-Benchmark-Feed**: öffentliches Branchen-Kennzahlen-Benchmark (KGK / TÜİK / Findeks) — heute nur Peer-Median
- [ ] **Benachrichtigungskanäle**: E-Mail + Slack + Discord — Hochrisiko-Alerts
- [ ] **Multi-Currency**: heute alles TRY — FX-Snapshot-Tabelle für USD/EUR-Konten

9.3 Langfristig (3+ Monate)

- [ ] **ML-Schicht**: regelbasierter Score + optionales Logistic-Regression / Gradient-Boosting-Overlay (sobald genug Datensätze vorliegen)
- [ ] **Open Banking**: Pull der Kontobewegungen → automatisches Aging
- [ ] **Mobile-App**: PWA oder React Native — leseorientiert (Score-Ansicht, Alerts)
- [ ] **Self-Serve Tenant-Onboarding**: Marketplace-SaaS (heute Tenant-Anlage manuell)
- [ ] **GDPR/KVKK-Panel**: Datenexport, Löschanfrage, Retention-Policy
- [ ] **2FA**: TOTP (pyotp) — insbesondere für Admin-Rollen
- [ ] **Audit-Log-Export**: JSON-Stream für SIEM (Splunk, ELK)

---

10. Kontinuierliche Verbesserung — Umgesetzte Härtungen

Thema	Verbesserung	Status
`version`-Feld in docker-compose obsolet	Feld entfernt	✅
Kein Webhook-Retry	`_post_with_retry` — 3 Versuche + exponentielles Backoff (1s/2s/4s); Retry bei 5xx und Netzwerkfehlern, 4xx einmaliger Versuch	✅
Keine Log-Korrelations-ID	Middleware `request_id` — UUID16 pro Request, Header `X-Request-ID` + `g.request_id` Log-Bindung	✅
Audit-Log physisch gelöscht	Spalte `AuditLog.deleted_at` — Soft-Delete-Pattern (Compliance-Retention)	✅
Keine DB-Backup-Automation	`scripts/backup.sh` — erkennt Postgres/SQLite automatisch, gzip, Retention-Rotation; **Prod-Cron 03:00**	✅
Flask 3.0.0 CVE-2026-27205	Pin `flask>=3.1.3` + Upgrade von pip/wheel/setuptools im CI	✅
Rate-Limiter `memory://` zwischen Workern aufgeteilt	`LIMITER_STORAGE_URI=redis://localhost:6379/1` — gemeinsamer Zähler, in Prod verifiziert (11. Falsch-Login lieferte 429)	✅
Führungsbewertung blieb auf TR eingefroren, wenn der Bericht in einer anderen Sprache geöffnet wurde	`assessment_i18n` / `decision_summary_i18n` — als JSON-Dict in 4 Sprachen gespeichert; View fällt auf aktive Sprache; Legacy-Zeilen abwärtskompatibel	✅
Excel-Import setzte `user_id` für neue `Customer`-Zeilen nicht	`current_user.id`-Zuweisung — verwaiste Datensätze behoben	✅
`Excel-Upload` im Menü auch für Nicht-Admins sichtbar (Klick → Redirect)	`is_admin`-Flag im `context_processor` + `{% if is_admin %}`-Gating; redundanter `Panel`-Link entfernt	✅
Kein Alarm bei 5xx	`errorhandler(500)` → Mail an `ADMIN_EMAIL` mit request_id + Pfad + Nutzer + Exception-Zusammenfassung	✅

Offen (auf der Roadmap)

Thema	Auswirkung	Plan
Kein Alembic	Leichtes ALTER reicht; wieder aufgreifen, wenn Schema wächst	Wenn das Schema 30+ Tabellen überschreitet
Geringe Test-Coverage	Refactor-Risiko	Pytest-Skelett vorhanden, Ziel 50 % auf kritischen Pfaden
`translations.py` ist eine 70-KB-Datei	Merge-Conflict-Magnet	Eine Datei pro Sprache + Lazy-Load
Authlib-Deprecation (`authlib.jose`)	Bricht in 2.0	Auf `joserfc` migrieren
Server-seitige Override-docker-compose	`git stash` bei jedem Pull	In `docker-compose.override.yml` aufteilen

---

11. Sicherheitsnotizen — Aktive Kontrollen

Identität & Session

- ✅ Passwörter via werkzeug PBKDF2-SHA256 (gesalzen)
- ✅ Mail-Verifikation verpflichtend (URLSafeTimedSerializer, befristet)
- ✅ Google OAuth — Authlib State-Validierung (CSRF-Schutz)
- ✅ Cookie-Flags: `HttpOnly` , `SameSite=Lax` , `Secure` in Prod

Web-Schicht (flask-wtf, flask-limiter, flask-talisman)

- ✅ **CSRF-Schutz**: `csrf_token` in jedem POST-Formular; API-Blueprint ausgenommen (token-basiert)
- ✅ **Rate-Limiting**: `/login` 10/min + 50/h, `/register` 5/min + 20/h; default 100/min + 1000/h. **Prod-Backend Redis** (`LIMITER_STORAGE_URI=redis://localhost:6379/1`) — Zähler über Worker geteilt.
- ✅ **Security-Header** (Talisman): HSTS (1 Jahr), `X-Frame-Options: SAMEORIGIN` , `X-Content-Type-Options: nosniff` , `Referrer-Policy: strict-origin-when-cross-origin`
- ✅ **Request-ID**: Jede Anfrage trägt einen `X-Request-ID` -Header → Log-Korrelation
- ✅ **Menü-Admin-Gating**: `is_admin` -Context-Flag blendet Links, die `@admin_required` benötigen, für Nicht-Admins aus

Datenzugriff

- ✅ Multi-Tenant-Isolation — jede Query filtert nach `tenant_id`

- ✅ SQLAlchemy-ORM — parametrisierte Queries (SQL-Injection-Schutz)
- ✅ `is_sample`-Flag umgeht die Tenant-Isolation nicht
- ✅ Postgres-Advisory-Lock — Schutz gegen Init-Race
- ✅ Audit-Log Soft-Delete — keine physische Löschung, Compliance-Retention

Operativ

- ✅ Geheimnisse in `.env`, nicht im Repo (`.gitignore` + 8 Sicherheits-Tests)
- ✅ DB-Backup-Skript + Retention-Rotation (`scripts/backup.sh`) — **Prod-Cron jede Nacht 03:00**
- ✅ Abhängigkeiten gepinnt, `pip-audit --strict` im CI (inkl. Flask-CVE-Bump)
- ✅ Upload-Cap 10 MB (DoS-Surface-Reduktion)
- ✅ **5xx-Admin-Alert**: `errorhandler(500)` mailt request_id + Exception an `ADMIN_EMAIL`

Roadmap (offen)

- ⚠️ 2FA (TOTP) — insbesondere Admin-Rollen
- ⚠️ Webhook-Outbound-HMAC-Signatur standardmäßig verpflichtend
- ⚠️ CSP (Content Security Policy) — heute Inline-Style/-Script, Refactor nötig
- ⚠️ Authlib-Migration auf `joserfc` (2.0-Deprecation)

---

12. Operative Befehle

```
# Lokale Entwicklung
python app.py                                    # SQLite

# Docker (lokaler Port 8005)
docker compose up -d --build
docker compose logs -f radar-app

# Prod-Deploy (von der lokalen Maschine)
sshpass -p "$SERVER_PASS" ssh "$SERVER_USER@$SERVER_IP" \
  "cd ~/apps/radar && git pull && \
   echo \$PASS | sudo -S docker compose up -d --build"

# Smoke-Test
curl -I https://technodai.com/radar/

# DB-Backup (in Prod)
sudo docker exec radar_v1_app pg_dump -U radar radar > backup_$(date +%F).sql

# Migration (leichtgewichtig, läuft automatisch in init_db)
python -c "from database import init_db; init_db()"

# Seed
python seed_sample_data.py

# Übersetzungs-Schlüssel-Parität (manuell)
python -c "from translations import translations; \
  ks = {l: set(translations[l]['expert_assessments'].keys()) for l in translations}
  print('diff:', ks['tr'] ^ ks['en'])"
```

13. Architekturentscheidungen (mini-ADR)

1. **Leichtes ALTER statt Alembic**: Schema klein, Downtime akzeptabel. Wieder prüfen, wenn die Komplexität wächst.
2. `network_mode: host` : gleicher Host wie nginx, keine Port-Mapping-Komplexität. Wird sich für Multi-Host ändern.
3. **flask-babel 4.x**: Flask 3.0 inkompatibel mit 3.x (`locked_cached_property` entfernt).
4. **Single-Tenant-Default-ID=1**: Alt-Datensätze rückwirkend gefüllt; Multi-Tenant-Übergang ohne Downtime.
5. **Postgres-Advisory-Lock-Schlüssel 7382001**: feste Magic-Number — alle Worker fordern dasselbe Lock an, der erste legt das Schema an, die übrigen warten.
6. **Sponsor-Pill im Footer**: in der Navbar war kein Platz, Sichtbarkeit wurde durch Verschiebung in den Footer erhalten.

14. Lizenz & Open Source

- **Projektlizenz:** AGPL-3.0 — wer es als Service hostet, muss den Quellcode teilen
- **Abhängigkeiten:** alle permissiv (MIT/BSD/Apache); kein GPL/AGPL → AGPL-kompatibel
- **Bemerkenswerte Abhängigkeiten:**
- `psycopg2-binary` (LGPL) — dynamisches Linken, kein Problem
- `certifi` (MPL-2.0) — Datei-Level-Copyleft, OK, solange wir es nicht modifizieren
- **Lizenz-Audit:** mit `pip-licenses` durchgeführt, Ausgabe sauber

15. Test & Qualität

Vorhandene Tests

- `tests/security_smoke.py` — 8 Sicherheits-Checks:
- `.env` und Geheimnisse nicht in git
- Keine Privatschlüssel-Dateien
- Keine hartkodierten SECRET/PASSWORD/API_KEY
- Kritische Pakete gepinnt (flask, sqlalchemy, authlib, itsdangerous, flasgger)
- Alle geschützten Routen mit `@login_required`
- SECRET_KEY gesetzt und lang
- LICENSE-Datei ist AGPL-3.0
- `tests/integrity_check.py` — manuelle Score-Validierung (DB nötig)

CI (GitHub Actions)

- `.github/workflows/ci.yml` — bei jedem Push/PR:
- pytest-Sicherheitssuite
- Abhängigkeits-Schwachstellen-Scan `pip-audit --strict`
- Übersetzungs-Schlüssel-Parität (TR/EN/ES/DE)

Ausführung

```
pytest tests/ -v                    # alle Tests
pytest -m security                  # nur Sicherheits-Smoke
pip-audit                           # CVE-Scan
```

16. Referenzen

- Repo: `git@github.com:Dai-Solutions/Radar.git`
- Prod: https://technodai.com/radar/
- Sponsor: OpenCollective + Patreon (`https://www.patreon.com/c/DynamicAI`)
- Findeks (Wettbewerber): https://www.findeks.com/